



Lab Title: Command Injection

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: BVWS103 – Owasp Top 10 Vulnerabilities
Exploitation Techniques

Instructor Name: AMMAR AMINU

Date of Submission: 25/01/2026

Executive Summary

This lab focused on understanding command injection and how it can affect a web application. I learned how unsafe handling of user input can allow attackers to run system commands on a server. The lab also showed how simple security measures, such as input sanitization, can help prevent these attacks and protect the system.

Lab Objective

The objectives of this lab were to:

- Understand what command injection is and how it works.
- Learn how attackers can exploit command injection vulnerabilities.
- Apply basic mitigation techniques to prevent command injection in web applications.

Lab Environment

The lab was performed in a controlled local environment using:

- HTML for creating the user input form.
- PHP for server-side scripting and command execution.
- A web browser to interact with the application.
- A local development setup to safely test both vulnerable and secured code.

This environment allowed safe testing of command injection attacks without affecting real systems.

Task 1: Exploit Command Injection Vulnerability

Step 1: Set Up the Vulnerable Web Application

- Created a new folder (Lab4_CommandInjection)

```

(raph@kali)-[/opt/lampp/htdocs]
$ mkdir Lab4_CommandInjection
mkdir: cannot create directory 'Lab4_CommandInjection': Permission denied

(raph@kali)-[/opt/lampp/htdocs]
$ sudo mkdir Lab4_CommandInjection
[sudo] password for raph:

(raph@kali)-[/opt/lampp/htdocs]
$ ls
applications.html  dashboard  img  Lab2_HTMLInjection
bitnami.css        favicon.ico index.php Lab4_CommandInjection

(raph@kali)-[/opt/lampp/htdocs]
$ sudo /opt/lampp/lampp start
Starting XAMPP for Linux 8.2.4-0 ...
XAMPP: Starting Apache ... already running.
XAMPP: Starting MySQL ... already running.
XAMPP: Starting ProFTPD ... already running.

```

- Created two new file (index.html & command.php) inside the folder

```

(raph@kali)-[/opt/lampp/htdocs]
$ cd Lab4_CommandInjection

(raph@kali)-[/opt/lampp/htdocs/Lab4_CommandInjection]
$ sudo nano index.html

(raph@kali)-[/opt/lampp/htdocs/Lab4_CommandInjection]
$ sudo nano command.php
[sudo] password for raph:

(raph@kali)-[/opt/lampp/htdocs/Lab4_CommandInjection]
$ ls
command.php  index.html

(raph@kali)-[/opt/lampp/htdocs/Lab4_CommandInjection]
$

```

Step 2: Implemented the Vulnerable Code

- **Index.html**

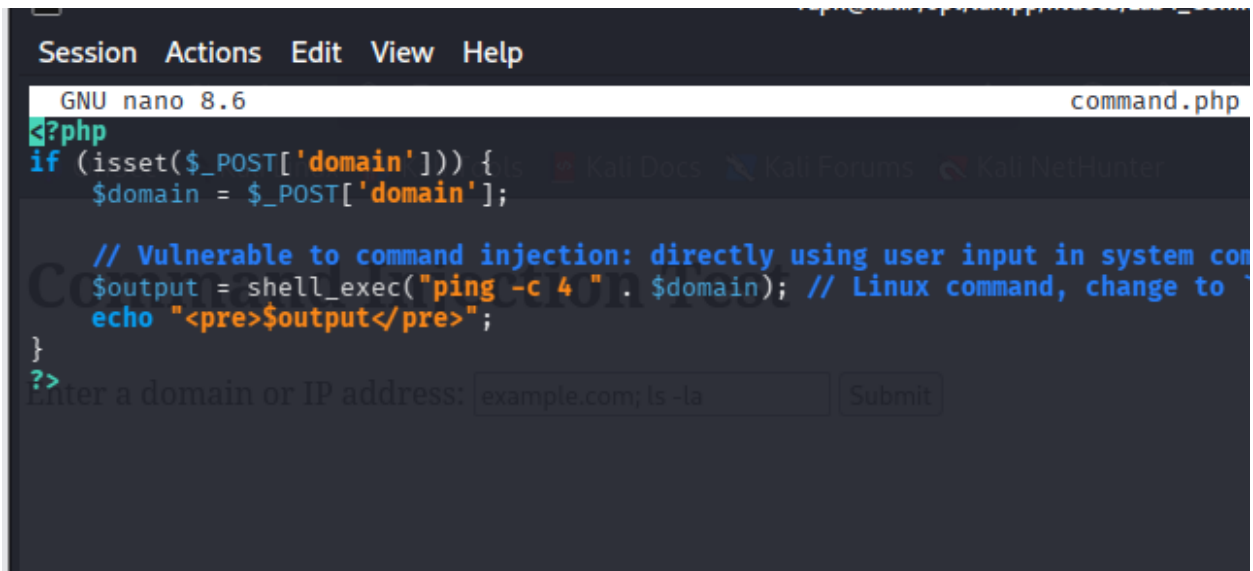
I created this file to display a simple form where I could enter a domain name or IP address. The form sends the user input to `command.php` using the POST method. This file helped me understand how user input is collected from a webpage and sent to the server for processing.

```
Session Actions Edit View Help
GNU nano 8.6 index.
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Command Injection Example</title>
  </head>
  <body>
    <h1>Command Injection Test</h1>
    <form action="command.php" method="post">
      <label for="domain">Enter a domain or IP address:</label>
      <input type="text" id="domain" name="domain">
      <button type="submit">Submit</button>
    </form>
  </body>
</html>
```

- **Command.php**

This file contains the PHP code that executes a system command based on the input provided by the user.

- In the **vulnerable version**, I directly used the user input in the ping command without any validation. Because of this, I was able to inject extra commands using special characters.
- In the **secured version**, I used the `escapeshellcmd()` function to sanitize the input before executing the command. This stopped command chaining and prevented command injection.



```
Session Actions Edit View Help
GNU nano 8.6 command.php
<?php
if (isset($_POST['domain'])) {
    $domain = $_POST['domain'];

    // Vulnerable to command injection: directly using user input in system command
    $output = shell_exec("ping -c 4 " . $domain); // Linux command, change to 'ping' for Windows
    echo "<pre>$output</pre>";
}
?>
```

Enter a domain or IP address:

Step 3: Exploiting the Vulnerability

I exploited the vulnerability by entering `example.com; ls -la` into the input field. The semicolon allowed me to execute another command after the ping command, which resulted in a directory listing being displayed. This showed that arbitrary commands could be executed on the server.

 http://localhost/Lab4_CommandInjection

 OffSec  Kali Linux  Kali Tools  Kali Docs  Kali Forums

Command Injection Test

Enter a domain or IP address:

 OffSec  Kali Linux  Kali Tools  Kali Docs  Kali Forums

```
PING example.com (104.18.27.120) 56(84) bytes of data.  
64 bytes from 104.18.27.120: icmp_seq=1 ttl=128 time=148 ms  
64 bytes from 104.18.27.120: icmp_seq=2 ttl=128 time=132 ms  
64 bytes from 104.18.27.120: icmp_seq=3 ttl=128 time=191 ms  
64 bytes from 104.18.27.120: icmp_seq=4 ttl=128 time=171 ms  
  
--- example.com ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3013ms  
rtt min/avg/max/mdev = 131.727/160.434/190.553/22.330 ms  
total 16  
drwxr-xr-x 2 root root 4096 Jan 27 14:59 .  
drwxr-xr-x 9 root root 4096 Jan 27 14:37 ..  
-rw-r--r-- 1 root root 291 Jan 27 14:59 command.php  
-rw-r--r-- 1 root root 530 Jan 27 14:40 index.html
```

Reflection Questions:

- **How does the ; symbol allow an attacker to chain commands together?**

The ; symbol tells the system to run another command after the first one finishes, so an attacker can add extra commands to be executed.

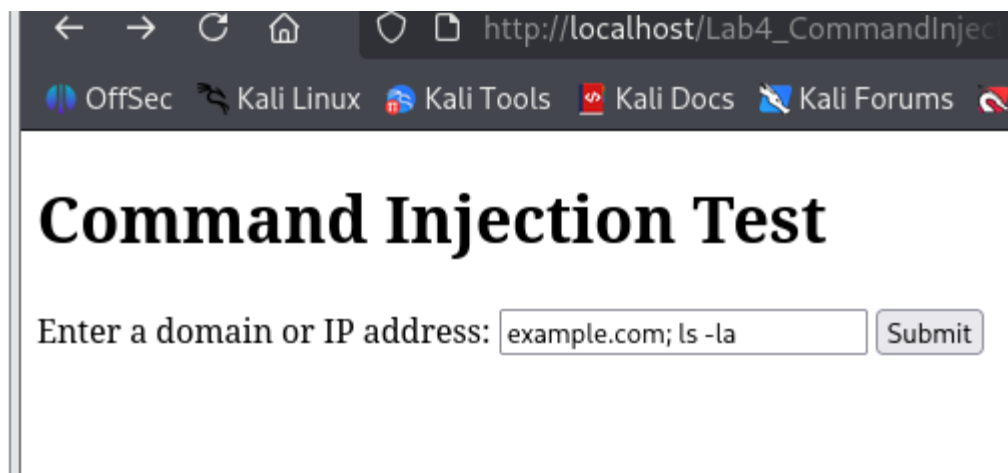
- **What risks are involved when web applications execute system commands based on user input?**

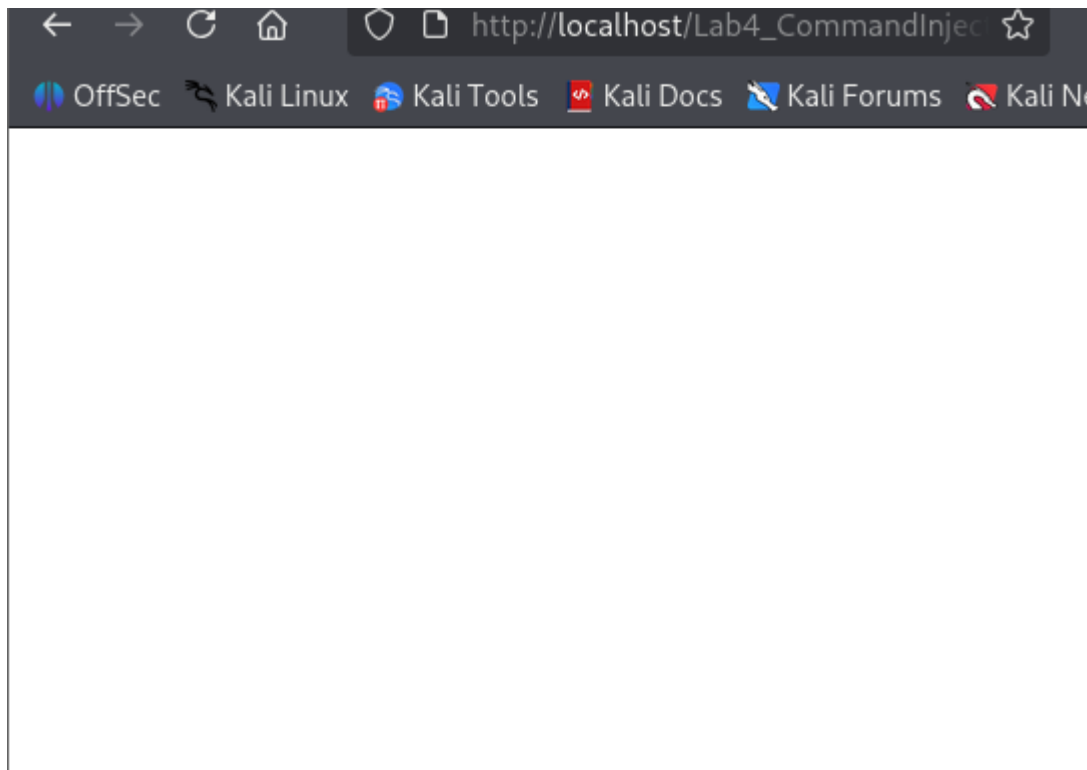
It can let attackers run harmful commands, access sensitive files, or take control of the server. Using command injection, an attacker can execute arbitrary system commands on the server.

Task 2: Mitigate Command Injection Vulnerability

Step 1: Fix the Vulnerability

I fixed the issue by using the `escapeshellcmd()` function to clean the user input. This function removes special characters that could be used for command injection. After applying this fix, the injected command no longer worked.





Importance of Input Validation

From this lab, I learned that trusting user input is risky. Input validation is very important because it helps protect web applications from attacks like command injection. This lab showed me how a small mistake can lead to serious security issues.

Reflection Questions:

- **How does the `escapshellcmd()` function prevent command injection?**

It blocks or escapes special characters so the input cannot be used to add extra system commands.

- **Why is it important to sanitize user input before using it in system commands?**

Because unsafe input can allow attackers to run harmful commands and take control of the system.

- **Can you think of other ways to secure this functionality?**

You can use input whitelisting, avoid system commands when possible, limit user permissions, and use safer built-in functions instead of shell commands.

Conclusion

In this lab, I learned how command injection vulnerabilities occur and how easily they can be exploited when user input is not properly handled. By exploiting the vulnerable code and then fixing it using input sanitization, I was able to see the importance of validating user input. This lab helped me understand how attackers think and how simple security measures can be used to protect web applications.